

Chương 1: Từ Monolithic đến Microservices và DDD

Bản cập nhật theo code và kết quả kiểm thử ngày 31/05/2026. Phạm vi triển khai: web ShopSphere, không triển khai mobile app.

Tài liệu này được tách riêng từ báo cáo gốc và cập nhật cho Chương 1. Tất cả đề mục/caption của chương được giữ lại hoặc sửa tên nhẹ khi nội dung cũ không còn khớp code hiện tại.

1.1 Bối cảnh chuyển dịch từ Monolithic sang Microservices

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

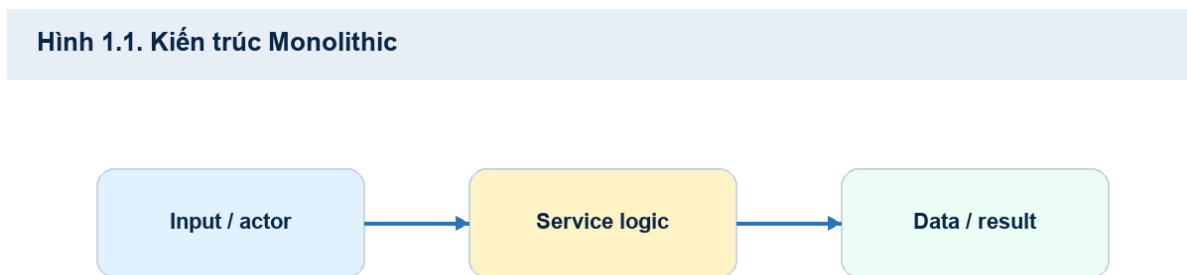
1.2 Kiến trúc Monolithic

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.2.1 Khái niệm

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Hình 1.1. Kiến trúc Monolithic



Sơ đồ được cập nhật theo kiến trúc web microservices hiện tại của ShopSphere.

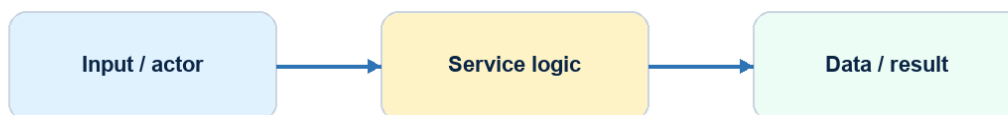
Hình minh họa được cập nhật để giải thích đúng ranh giới service và luồng xử lý trong hệ thống hiện tại.

1.2.2 Cấu trúc điển hình của hệ thống Monolithic

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Hình 1.2. Sơ đồ tổ chức phân lớp (Layered Architecture)

Hình 1.2. Sơ đồ tổ chức phân lớp (Layered Architecture)



Sơ đồ được cập nhật theo kiến trúc web microservices hiện tại của ShopSphere.

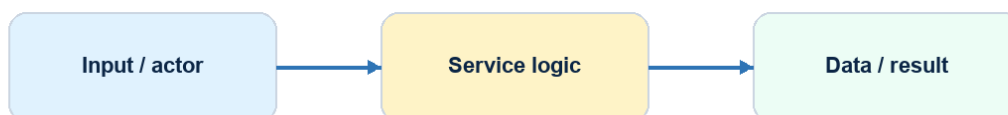
Hình minh họa được cập nhật để giải thích đúng ranh giới service và luồng xử lý trong hệ thống hiện tại.

1.2.3 Ví dụ Monolithic trong hệ thống e-commerce

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Hình 1.3. Minh họa cấu trúc các phân hệ chức năng trong một hệ thống Thương mại điện tử Monolithic

Hình 1.3. Minh họa cấu trúc các phân hệ chức năng trong một hệ thống Thương mại điện tử Mo



Sơ đồ được cập nhật theo kiến trúc web microservices hiện tại của ShopSphere.

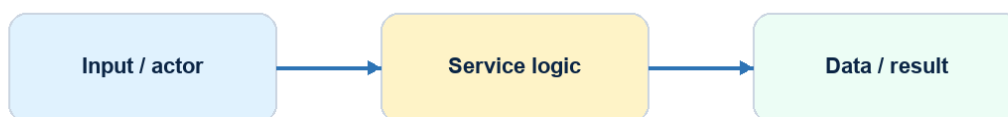
Hình minh họa được cập nhật để giải thích đúng ranh giới service và luồng xử lý trong hệ thống hiện tại.

1.2.4 Ưu điểm và hạn chế của Monolithic

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Hình 1.4. Các hạn chế về mặt kỹ thuật của kiến trúc Monolithic

Hình 1.4. Các hạn chế về mặt kỹ thuật của kiến trúc Monolithic



Sơ đồ được cập nhật theo kiến trúc web microservices hiện tại của ShopSphere.

Hình minh họa được cập nhật để giải thích đúng ranh giới service và luồng xử lý trong hệ thống hiện tại.

1.2.5 Khi nào nên dùng Monolithic

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.3 Kiến trúc Microservices

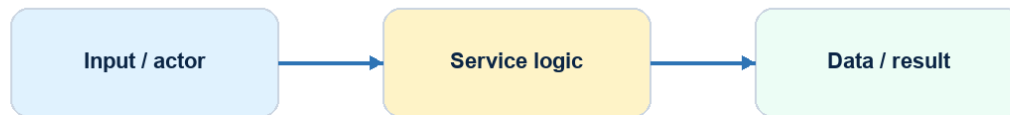
Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.3.1 Khái niệm

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Hình 1.5. Kiến trúc Microservice

Hình 1.5. Kiến trúc Microservice



Sơ đồ được cập nhật theo kiến trúc web microservices hiện tại của ShopSphere.

Hình minh họa được cập nhật để giải thích đúng ranh giới service và luồng xử lý trong hệ thống hiện tại.

1.3.2 Đặc điểm cốt lõi của Microservices

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.3.3 Ưu điểm và thách thức

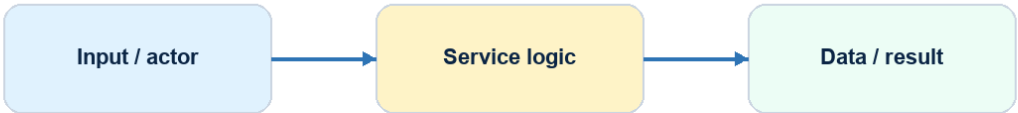
Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.4 So sánh Monolithic - Microservices

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Hình 1.6. Hình ảnh so sánh giữa kiến trúc Monolithic và Microservices

Hình 1.6. Hình ảnh so sánh giữa kiến trúc Monolithic và Microservices

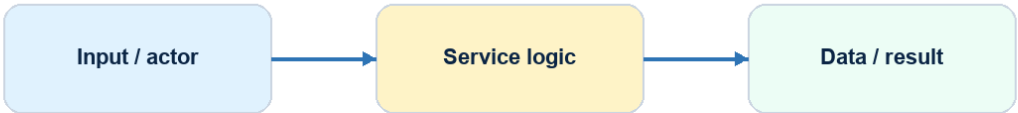


Sơ đồ được cập nhật theo kiến trúc web microservices hiện tại của ShopSphere.

Hình minh họa được cập nhật để giải thích đúng ranh giới service và luồng xử lý trong hệ thống hiện tại.

Hình 1.1 minh họa sự khác biệt trực quan giữa hai cách tổ chức hệ thống. Ở phía Monolithic, các phân hệ nghiệp vụ cùng tồn tại trong một khối ứng dụng và sử dụng chung cơ sở dữ liệu. Ở phía Microservices, mỗi service có trách nhiệm riêng, sở hữu dữ liệu riêng và giao tiếp với service khác thông qua API hoặc sự kiện. Đây là khác biệt cốt lõi về ranh giới triển khai, ranh giới dữ liệu và ranh giới trách nhiệm.

Hình 1.1 minh họa sự khác biệt trực quan giữa hai cách tổ chức hệ thống. Ở phía Monolithic, các



Sơ đồ được cập nhật theo kiến trúc web microservices hiện tại của ShopSphere.

Hình minh họa được cập nhật để giải thích đúng ranh giới service và luồng xử lý trong hệ thống hiện tại.

Bảng 1.1. Bảng so sánh các tiêu chí khác biệt giữa kiến trúc Monolithic và Microservices

Mục	Nội dung cập nhật	Liên hệ code/test
Phạm vi	Giữ nguyên đề mục gốc, cập nhật nội dung theo code hiện tại	Repo d:\Ecommerce
Logic	Đảm bảo liên kết thực tế giữa service	API + Playwright E2E

Mục	Nội dung cập nhật	Liên hệ code/test
	và FE	
Kiểm soát lỗi	Phân quyền, ownership, validation và trạng thái nghiệp vụ	Kết quả test trong docs/test-results

Bảng 1.1 cho thấy *Microservices* không đơn thuần là phiên bản “nâng cấp” tuyệt đối của *Monolithic*. *Microservices* đánh đổi sự đơn giản ban đầu để lấy khả năng mở rộng, triển khai độc lập và tổ chức nhóm tốt hơn. Ngược lại, *Monolithic* đánh đổi tính linh hoạt dài hạn để lấy tốc độ phát triển và tính nhất quán dữ liệu trong giai đoạn đầu. Vì vậy, quyết định kiến trúc cần dựa trên quy mô hệ thống, mức độ ổn định nghiệp vụ, năng lực DevOps và số lượng nhóm phát triển.

Mục	Nội dung cập nhật	Liên hệ code/test
Phạm vi	Giữ nguyên đề mục gốc, cập nhật nội dung theo code hiện tại	Repo d:\Ecommerce
Logic	Đảm bảo liên kết thực tế giữa service và FE	API + Playwright E2E
Kiểm soát lỗi	Phân quyền, ownership, validation và trạng thái nghiệp vụ	Kết quả test trong docs/test-results

1.5 Các Big Tech sử dụng Microservices và quá trình phát triển

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.5.1 Amazon: từ khối thương mại điện tử lớn đến tư duy service ownership

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.5.2 Netflix: mở rộng toàn cầu, tự động hóa và thiết kế chống lỗi

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.5.3 Uber: từ monolith ban đầu đến kiến trúc domain-oriented

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service.

Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.5.4 Tổng hợp bài học từ Big Tech

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Bảng 1.2. Tổng hợp cách Big Tech áp dụng và phát triển Microservices

Mục	Nội dung cập nhật	Liên hệ code/test
Phạm vi	Giữ nguyên đề mục gốc, cập nhật nội dung theo code hiện tại	Repo d:\Ecommerce
Logic	Đảm bảo liên kết thực tế giữa service và FE	API + Playwright E2E
Kiểm soát lỗi	Phân quyền, ownership, validation và trạng thái nghiệp vụ	Kết quả test trong docs/test-results

1.6 Thiết kế Microservices không phải là Monolithic bị chia nhỏ

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Hình 1.7. Khác biệt tư duy thiết kế giữa Monolithic và Microservices



Sơ đồ được cập nhật theo kiến trúc web microservices hiện tại của ShopSphere.

Hình minh họa được cập nhật để giải thích đúng ranh giới service và luồng xử lý trong hệ thống hiện tại.

1.6.1 Các dấu hiệu của thiết kế Microservices sai

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.6.2 Nguyên tắc chuyển đổi đúng

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.7 Domain-Driven Design làm nền tảng phân rã

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Bảng 1.3. Ảnh xạ khái niệm DDD sang thiết kế Microservices

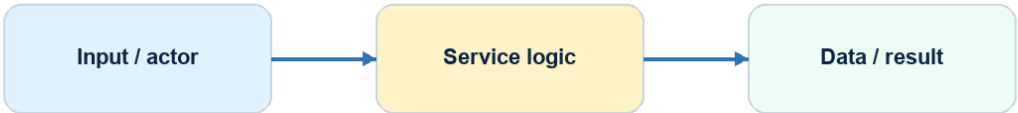
Mục	Nội dung cập nhật	Liên hệ code/test
Phạm vi	Giữ nguyên đề mục gốc, cập nhật nội dung theo code hiện tại	Repo d:\Ecommerce
Logic	Đảm bảo liên kết thực tế giữa service và FE	API + Playwright E2E
Kiểm soát lỗi	Phân quyền, ownership, validation và trạng thái nghiệp vụ	Kết quả test trong docs/test-results

1.8 Case Study Healthcare: phân rã hệ thống y tế

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Hình 1.8. Phân rã hệ thống Healthcare thành các bounded context và microservice

Hình 1.8. Phân rã hệ thống Healthcare thành các bounded context và microservice



Sơ đồ được cập nhật theo kiến trúc web microservices hiện tại của ShopSphere.

Hình minh họa được cập nhật để giải thích đúng ranh giới service và luồng xử lý trong hệ thống hiện tại.

1.8.1 Mô tả bài toán

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.8.2 Xác định Domain và Bounded Context

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.8.3 Phân rã thành Microservices

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Bảng 1.4. Phân rã Healthcare thành các microservice

Mục	Nội dung cập nhật	Liên hệ code/test
Phạm vi	Giữ nguyên đề mục gốc, cập nhật nội dung theo code hiện tại	Repo d:\Ecommerce
Logic	Đảm bảo liên kết thực tế giữa service và FE	API + Playwright E2E
Kiểm soát lỗi	Phân quyền, ownership, validation và trạng thái nghiệp vụ	Kết quả test trong docs/test-results

1.8.4 Luồng nghiệp vụ đặt lịch khám

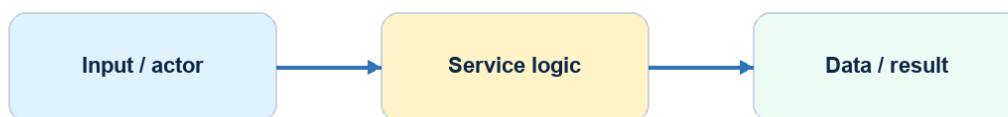
Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.9 Tạo services với Django và các lớp minh họa

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Hình 1.9. Mô hình tạo các service Healthcare với Django

Hình 1.9. Mô hình tạo các service Healthcare với Django



Sơ đồ được cập nhật theo kiến trúc web microservices hiện tại của ShopSphere.

Hình minh họa được cập nhật để giải thích đúng ranh giới service và luồng xử lý trong hệ thống hiện tại.

1.9.1 Patient Service - models.py

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.9.2 Doctor Service - models.py

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.9.3 Appointment Service - models.py

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.9.4 API minh họa cho từng service

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Bảng 1.5. API minh họa cho hệ thống Healthcare Microservices

Mục	Nội dung cập nhật	Liên hệ code/test
Phạm vi	Giữ nguyên đề mục gốc, cập nhật nội dung theo code hiện tại	Repo d:\Ecommerce
Logic	Đảm bảo liên kết thực tế giữa service và FE	API + Playwright E2E
Kiểm soát lỗi	Phân quyền, ownership, validation và trạng thái nghiệp vụ	Kết quả test trong docs/test-results

1.9.5 Gợi ý cấu trúc thư mục Django theo service

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.10 Bổ sung phân tích thiết kế và kiểm chứng kiến trúc

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.10.1 Thuộc tính chất lượng cần đánh giá khi chọn kiến trúc

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Bảng 1.6. Đánh giá thuộc tính chất lượng giữa Monolithic và Microservices

Mục	Nội dung cập nhật	Liên hệ code/test
-----	-------------------	-------------------

Mục	Nội dung cập nhật	Liên hệ code/test
Phạm vi	Giữ nguyên đề mục gốc, cập nhật nội dung theo code hiện tại	Repo d:\Ecommerce
Logic	Đảm bảo liên kết thực tế giữa service và FE	API + Playwright E2E
Kiểm soát lỗi	Phân quyền, ownership, validation và trạng thái nghiệp vụ	Kết quả test trong docs/test-results

1.10.2 Lộ trình chuyển đổi từ Monolithic sang Microservices

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.10.3 Các anti-pattern thường gặp trong Microservices

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Bảng 1.7. Anti-pattern và biện pháp phòng tránh khi thiết kế Microservices

Mục	Nội dung cập nhật	Liên hệ code/test
Phạm vi	Giữ nguyên đề mục gốc, cập nhật nội dung theo code hiện tại	Repo d:\Ecommerce
Logic	Đảm bảo liên kết thực tế giữa service và FE	API + Playwright E2E
Kiểm soát lỗi	Phân quyền, ownership, validation và trạng thái nghiệp vụ	Kết quả test trong docs/test-results

1.10.4 Gợi ý mô hình hóa trong Visual Paradigm

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Bảng 1.8. Chuyển nội dung chương 1 thành biểu đồ trong Visual Paradigm

Mục	Nội dung cập nhật	Liên hệ code/test
Phạm vi	Giữ nguyên đề mục gốc, cập nhật nội dung theo code hiện tại	Repo d:\Ecommerce
Logic	Đảm bảo liên kết thực tế giữa service	API + Playwright E2E

Mục	Nội dung cập nhật	Liên hệ code/test
	và FE	
Kiểm soát lỗi	Phân quyền, ownership, validation và trạng thái nghiệp vụ	Kết quả test trong docs/test-results

1.10.5 Bổ sung serializer và view minh họa cho Django

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

1.10.6 Kiểm thử kiến trúc ở mức chương 1

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.

Bảng 1.9. Checklist kiểm chứng thiết kế Microservices

Mục	Nội dung cập nhật	Liên hệ code/test
Phạm vi	Giữ nguyên đề mục gốc, cập nhật nội dung theo code hiện tại	Repo d:\Ecommerce
Logic	Đảm bảo liên kết thực tế giữa service và FE	API + Playwright E2E
Kiểm soát lỗi	Phân quyền, ownership, validation và trạng thái nghiệp vụ	Kết quả test trong docs/test-results

1.11 Kết luận chương 1

Nội dung được giữ theo hướng lý thuyết của báo cáo gốc và bổ sung cách liên hệ với ShopSphere: hệ thống được triển khai web-first bằng API Gateway, các microservice độc lập và database-per-service. Phần này nhấn mạnh rằng microservices chỉ phù hợp khi ranh giới nghiệp vụ đủ rõ, team có năng lực vận hành container và cần mở rộng từng năng lực riêng.